AIR UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

EXPERIMENT NO 12

Lab Title: Finite State Machine
Student Name: Faiq Ali_____ **Reg. No:** 249520

Objective: They are particularly useful for systems that require a specific sequence of operations or responses to events. The system moves from one state to another based on defined **inputs** or **events**

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					

Conclusion					

Total Marks: _____ Obtained Marks:

_____ Date: _____

Signature:

_____

Digital Logic Design (EE-222) Department of Electrical Engineering

EXPERIMENT 12

Design of Finite State Machine

Objectives:

- Design a circuit that detects a sequence of three or more consecutive 1'sin a string of bits coming through an input line(i.e. the input is a serial bitstream) **Required Components and Equipment:**

- Basic Gate ICs
- Digital Electronics Trainer
- Bread Board
- Connecting Wires
- D-Flip Flop **Introduction:**

Up until now, every circuit that was presented was a combinatorial circuit. That means that its output is dependent only by its current inputs. Previous inputs for that type of circuits have no effect on the output.

However, there are many applications where there is a need for our circuits to have "memory"; to remember previous inputs and calculate their outputs according to them. A circuit whose output depends not only on the present input but also on the history of the input is called a sequential circuit.

In this section we will learn how to design and build such sequential circuits. In order to see how thisprocedure works, we will use an example, on which we will study our topic. In FSM

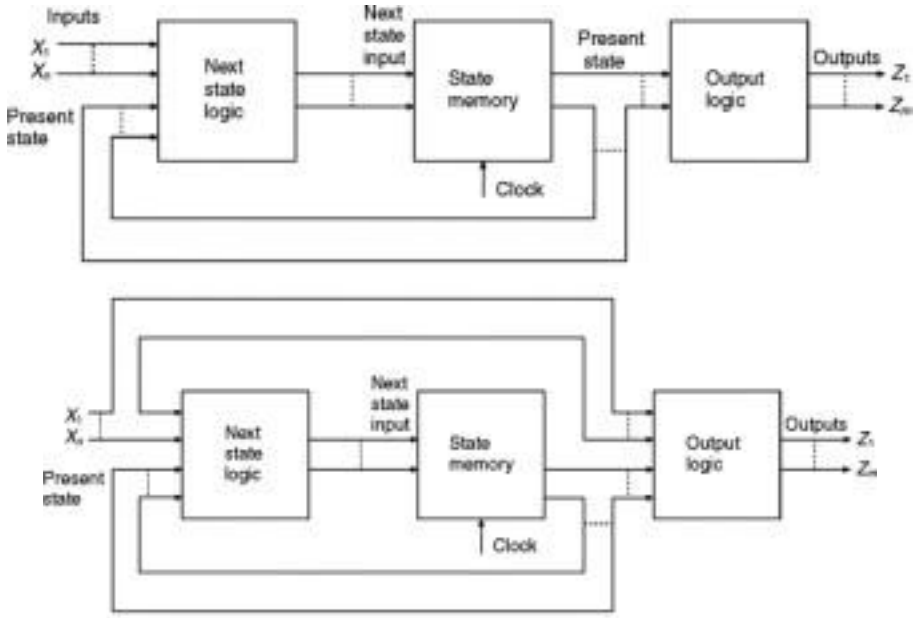
- Output of combinational logic depend on present valued of the input.
- So combinational logic devices lack memory elements.

• State machines possess memory and can perform time dependent sequences of logic signalscontrolled by present and past information.

• State machines are mostly used as a control logic.

• Usually number of states are limited so they are called finite state machine (FSM).

Mealy and Moore Machines



The design procedure has specific steps that must be followed in order to get the work done as:

- Specify the problem with word (e.g. Design a circuit that detects three consecutive 1 inputs)
- Assign binary values to states
- Develop a state table
- Use K-maps to simplify expressions
- Flip flop input equations and output equations
- Create appropriate logic diagram
- Should include combinational logic and flip flops

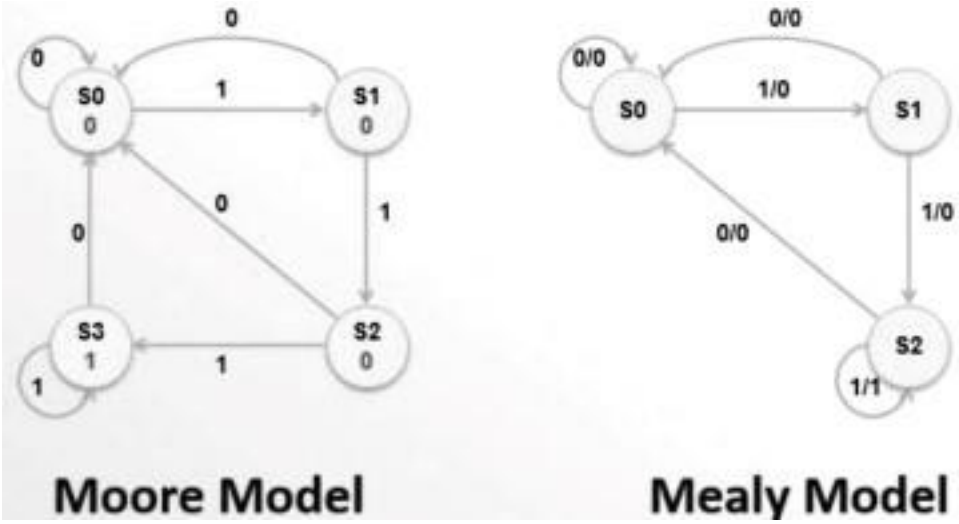
Mealy vs. Moore Machines

A key difference between Moore and Mealy machines is that in a Moore machine, the output is solely determined by the current state, while in a Mealy machine, the output depends on both the current state and the current input; essentially, Moore machines produce outputs based on their state alone, while Mealy machines produce outputs based on both state and input. **Lab Task 1**

Design a circuit that detects a sequence of three or more consecutive 1's in astring of bits coming through an input line (i.e. the input is a serial bit stream)

State Diagram

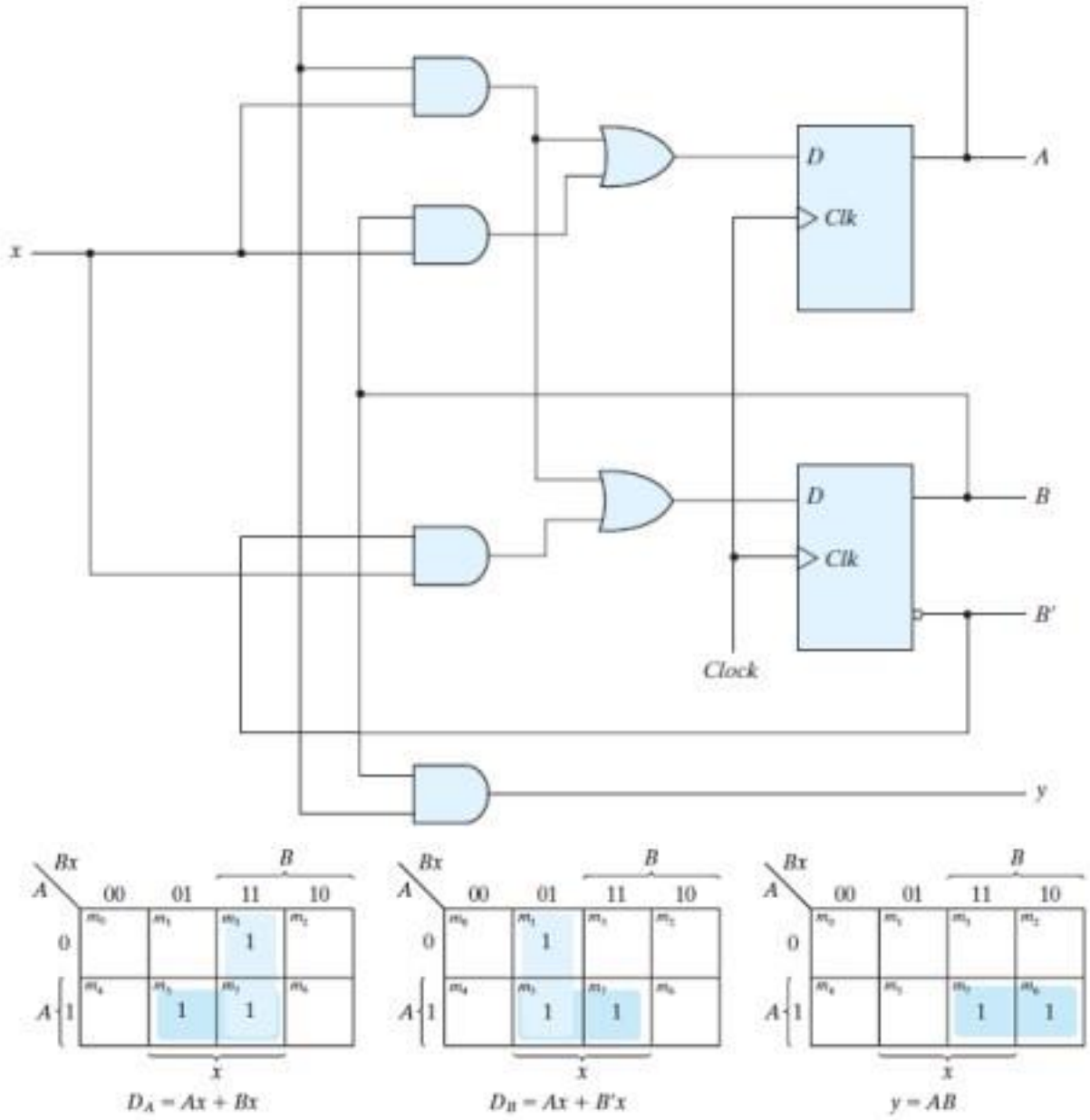
- **Note that each state has 2 output arrows**
- **Two bits needed to encode state**



Step1.Investigate state diagram and derive next state table from state diagram

Present State		Input	Next State		Output
A	*B*	*x*	*A*	*B*	*y*
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

Step 2. Derive the simplified Flip Flop input equations and output equationfrom K-Maps



Step 3. Create K-map directly from state table (3 columns = 3 K-maps)

Step 4. Minimize K-maps to find SOP representations. Separate circuit for each next state and output value

Lab Task 2

Design a **Finite State Machine (FSM)**–based control circuit for a **two-floor elevator**. Use **D flip flops**, **combinational logic**, and **Moore outputs**.

Inputs

- **Req1** – Request for Floor-1
- **Req2** – Request for Floor-2

- **FS1** – Floor-1 sensor (1 when elevator is at Floor-1)
- **FS2** – Floor-2 sensor (1 when elevator is at Floor-2) *Assume FS1 and FS2 are never 1 at the same time.*

Outputs (Moore)

- **MotorUp**

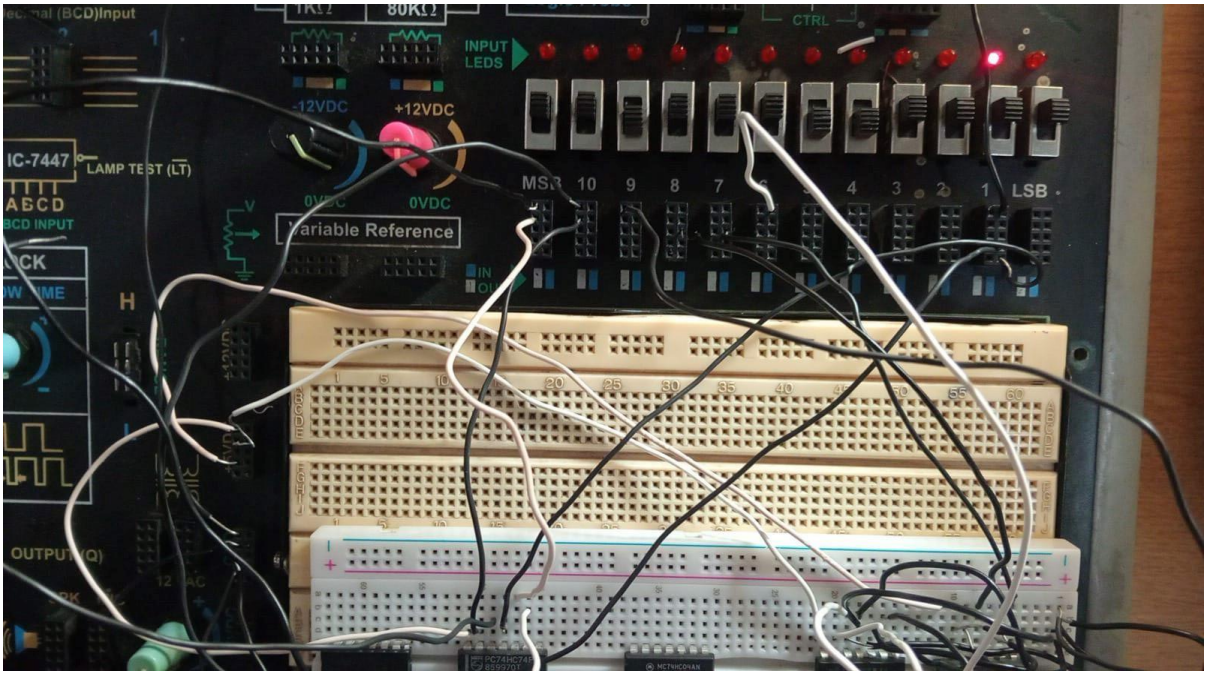
- **MotorDown**
- **DoorOpen**

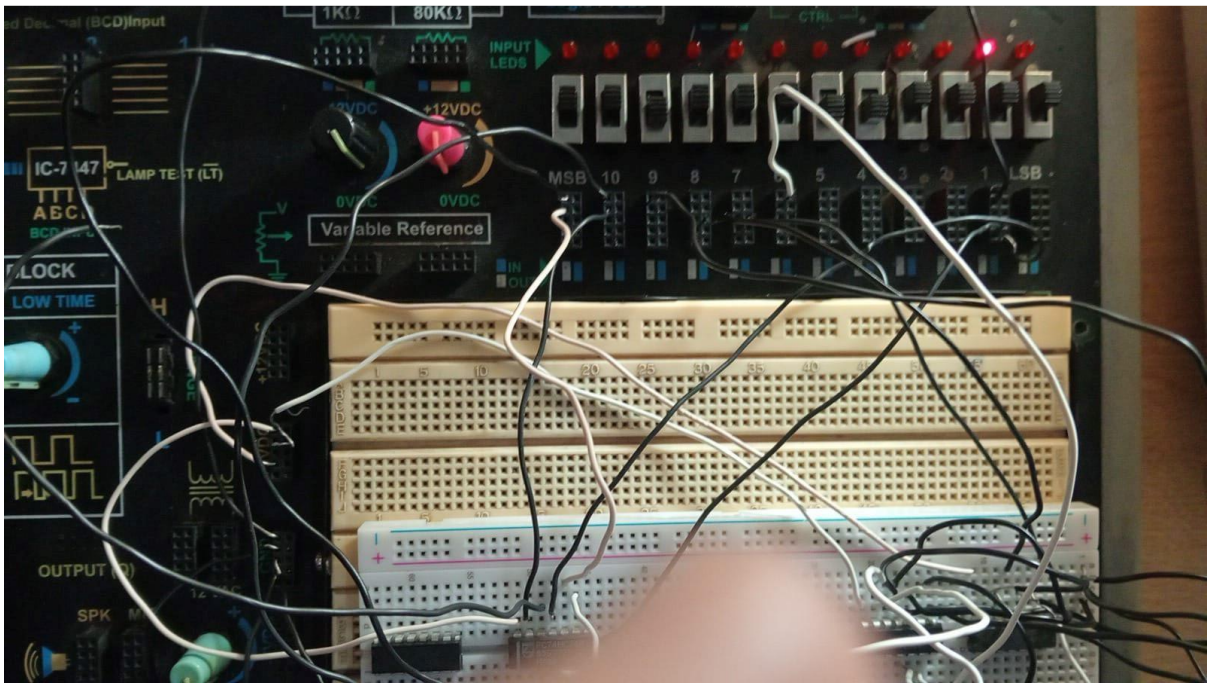
Required Elevator Behavior

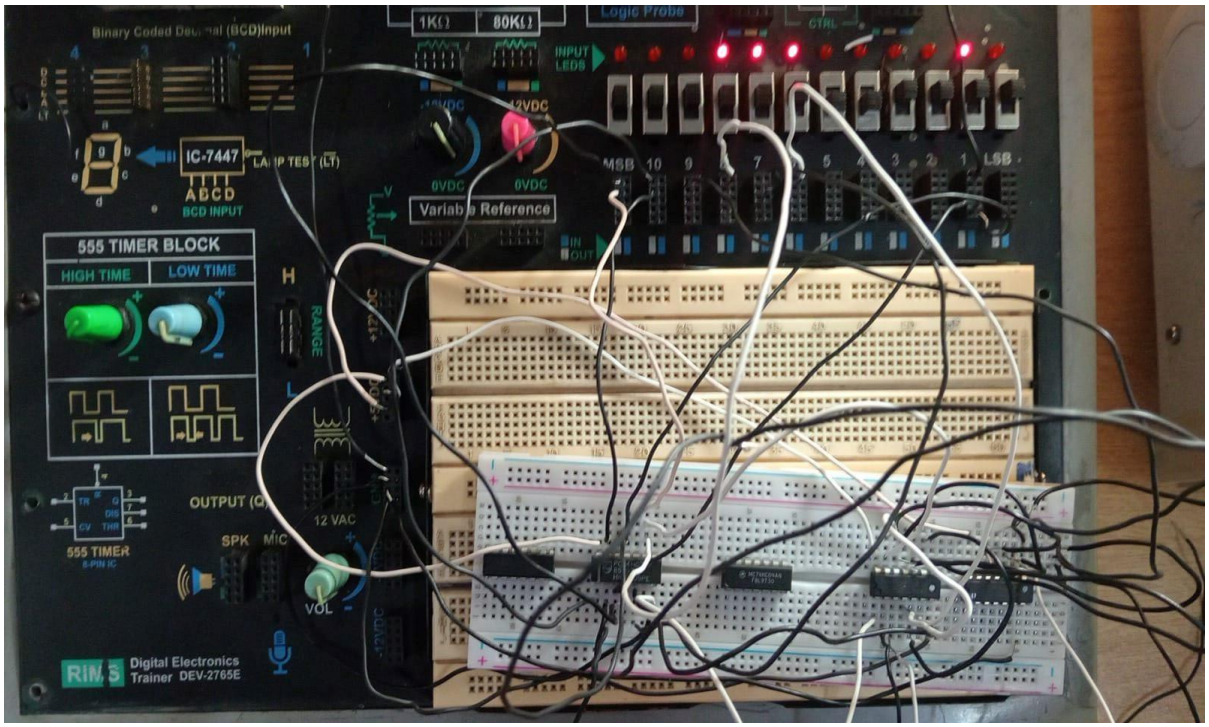
1. **Initial State:** Elevator starts at **Floor-1 idle** with the door open.
2. **Idle States:**
   - At Floor-1: If Req2 = 1 → move up; otherwise stay idle.
   - At Floor-2: If Req1 = 1 → move down; otherwise stay idle.
3. **Motion States:**
   - Moving Up: Continue until FS2 = 1 → then idle at Floor-2.
   - Moving Down: Continue until FS1 = 1 → then idle at Floor-1.
4. **Ignore same-floor requests** when already idle at that floor.

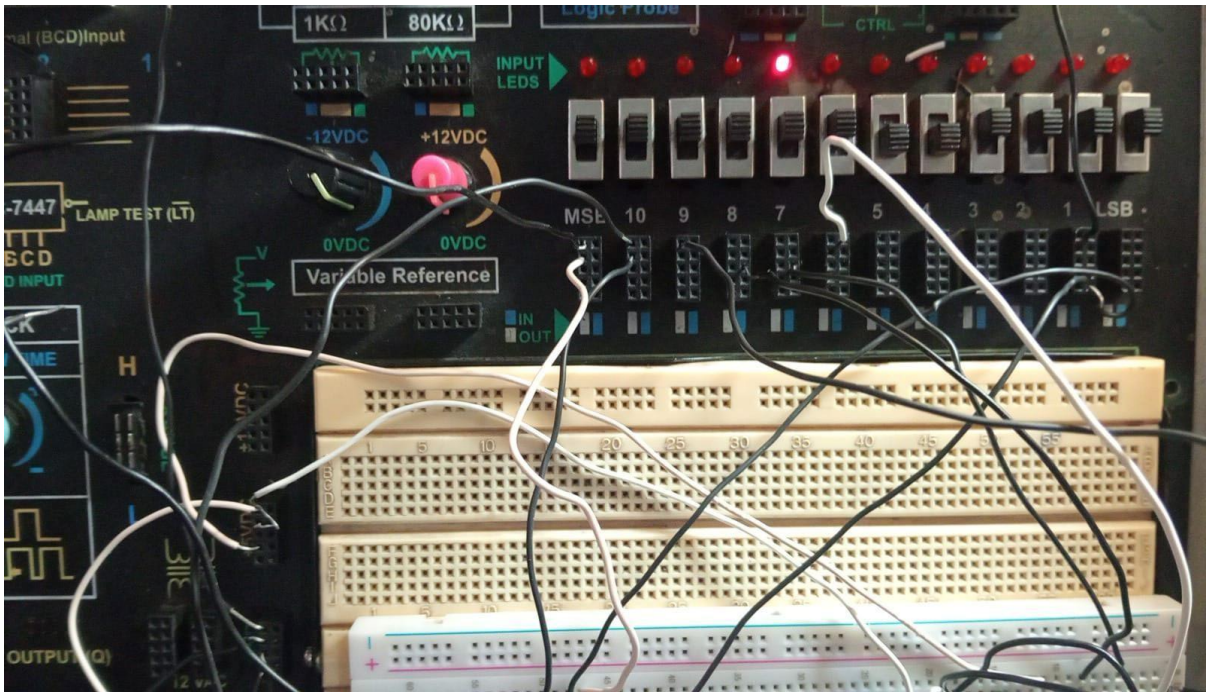
Design Task you must produce.

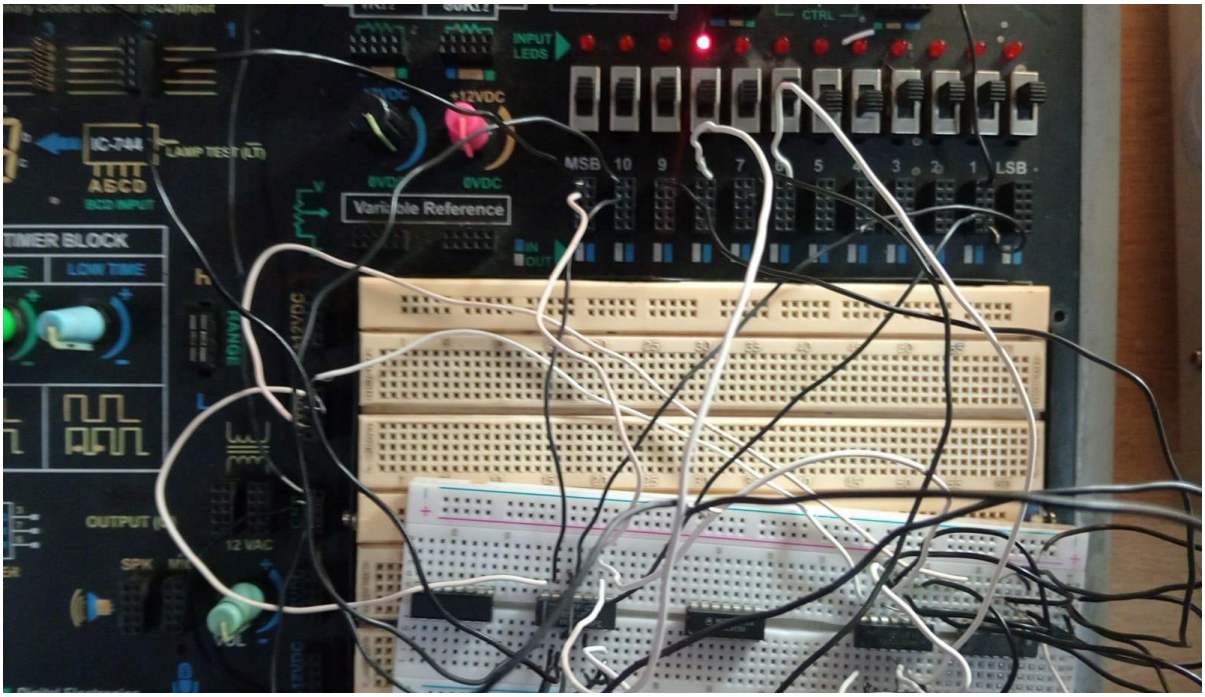
1. State diagram
2. State transition table
3. K-maps for next-state logic (D1, D0)
4. Simplified Boolean equations for D1, D0
5. Logic diagram using D-Flip Flops and gates




1KΩ
80KΩ
INPUT LEDS
+12VDC
0VDC
IC-7447
LAMP TEST (LT)
Variable Reference
MSB
LSB
LOW TIME
OUTPUT
SPK


Binary Coded Decimal (BCD)Input
1KΩ
80KΩ
Logic Probe
INPUT LEDS
+12VDC
0VDC
IC-7447
LAMP TEST (LT)
ABCD
BCD INPUT
Variable Reference
MSB
LSB
555 TIMER BLOCK
HIGH TIME
LOW TIME
RANGE
555 TIMER
8-PIN IC
12 VAC
SPK
MIC
VOL
Digital Electronics Trainer DEV-2765E





Design a Finite State Machine (FSM)–based control circuit for a two-floor elevator. Use D flip flops, combinational logic, and Moore outputs.

Inputs

- Req1 – Request for Floor-1
- Req2 – Request for Floor-2
- FS1 – Floor-1 sensor (1 when elevator is at Floor-1)
- FS2 – Floor-2 sensor (1 when elevator is at Floor-2)

Assume FS1 and FS2 are never 1 at the same time.

Outputs (Moore)
• MotorUp
Digital Logic Design (EE-222) Department of Electrical Engineering
• MotorDown
• DoorOpen

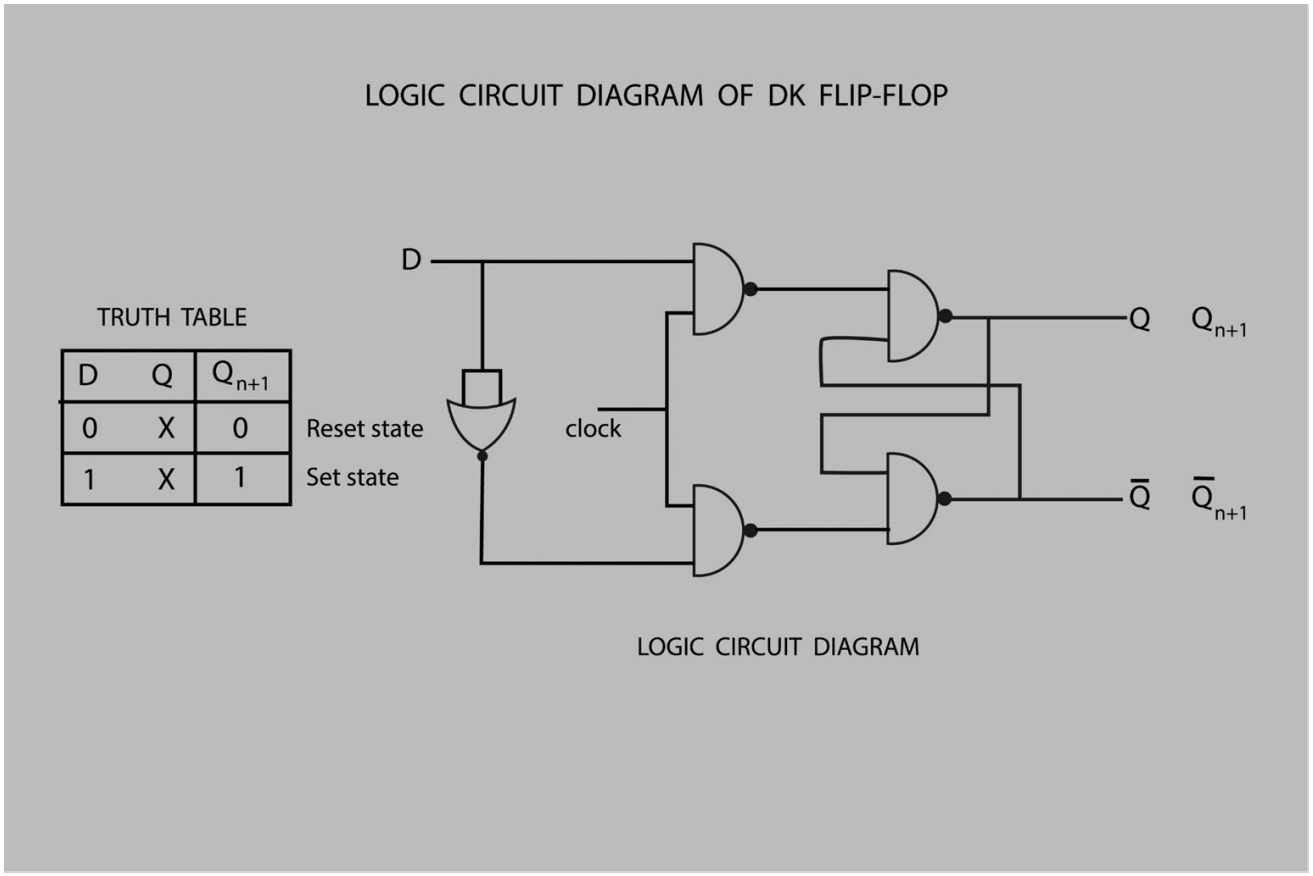
Required Elevator Behavior
1. Initial State: Elevator starts at Floor-1 idle with the door open. 2. Idle States:
o At Floor-1: If Req2 = 1 → move up; otherwise stay idle. o At Floor-2: If Req1 = 1 → move down; otherwise stay idle. 3. Motion States:
o Moving Up: Continue until FS2 = 1 → then idle at Floor-2. o Moving Down: Continue until FS1 = 1 → then idle at Floor-1. 4. Ignore same-floor requests when already idle at that floor.
Design a Task:
1. State diagram
2. State transition table
3. K-maps for next-state logic (D1, D0)
4. Simplified Boolean equations for D1, D0
5. Logic diagram using D-Flip Flops and gates

Solution:

State Name	State Encoding Q1 Q0	Output MUp MDown DO pen	Description
S1	0	0 0 1	Idle at Floor-1, Door Open
S2	1	0 0 1	Idle at Floor-2, Door Open
UP	10	1 0 0	Motor Moving Up, Door Closed
DOWN	11	0 1 0	Motor Moving Down, Door Closed

Q1 Q0		State	Req1 (R1)	Req2 (R2)	FS1 (F1)	FS2 (F2)	D1 D0		Next State
	0	S1	×	0	×	×		0	S1

	0	S1	×	1	×	×		10	UP
	1	S2	0	×	×	×		1	S2
	1	S2	1	×	×	×		11	DOWN
	10	UP	×	×	×		0	10	UP
	10	UP	×	×	×		1	1	S2
	11	DOWN	×	×		0 ×		11	DOWN
	11	DOWN	×	×		1 ×		0	S1



Conclusion:

1. Modeling Sequential Behavior: An FSM is a mathematical model of computation that defines how a system's output and next condition are determined by its current state and the incoming input. It is essential for designing sequential circuits (like the elevator controller) and software logic where the order and history of events matter.
2. Finite and Predictable Memory: The defining characteristic is that the system can only exist in one of a finite, predetermined number of

states at any given time. This limitation on "memory" (the states themselves) makes the FSM inherently predictable, simple to debug, and ideal for controlling specific, complex sequences (like protocols, parsers, or simple machine logic).